CSE 150: Introduction to Computer Networks

Chris Parsa

Chapter 3 Part 3

0

Current Assignments

- Lab6: pay attention to due dates in the assignment
  - 2 parts and in person demo with teaching team
- Group Assignment 5: next week
- Chapter 3 Practice
  - Posted are problems for Ch 3
  - End of Chapter problems for Congestion Control and TCP:
    - sections 3.5 - 3.7 (R14, R15, R16, R17, R18 and R19)
    - Extra problems at end of chapter as needed for practice
    - TCP problems on posted midterm exam
    - Make a study group – practice with friends!
  - IMPORTANT: Section 3.7 on sockets is required for the Project – TCP sockets, specifically
- Test 2 – Nov 19th

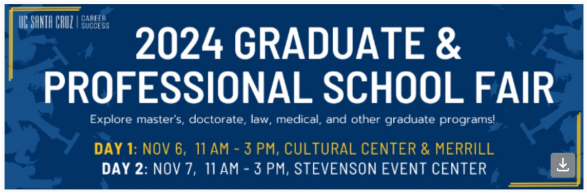
1

Final Project

- Partner – groups of 2 (requirement)
- Find your own partner this week
  - Sign up on Canvas soon
  - If you don't find your own, we will randomly assign

2

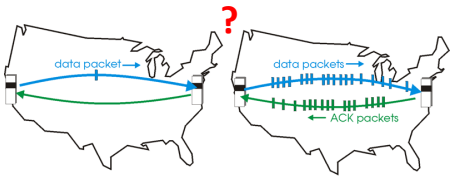
3



4

Pipelined protocols (ARQ Protocols)

Send multiple, "in-flight", yet-to-be-acknowledged packets



(a) a stop-and-wait protocol in operation (b) a pipelined protocol in operation

- Two generic forms of pipelined protocols: *Go-back-N, Selective Repeat* with fixed window
- *TCP – no fixed window!*

10

TCP: Transmission Control Protocol

- Reliable Data Delivery
  - Based on rdt 3.0
  - Reliable, in-order delivery with duplicate detection
- Tools to account for:
  - Bit errors
  - Positive feedback
  - Duplicate detection
  - Packet Loss

11

TCP: Overview

RFCs: 793, 1122, 1323, 2018, 2581

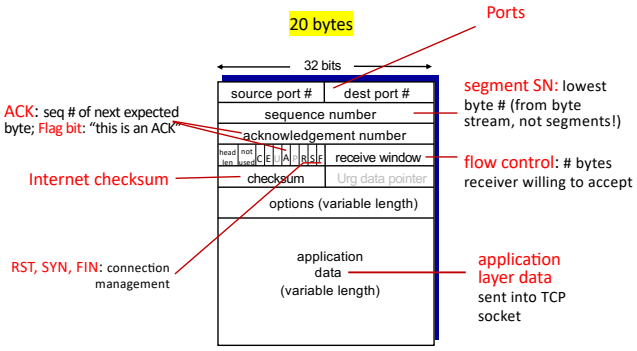
- Point-to-point:
  - One sender, one receiver
- Connection-oriented:
  - Handshake before data exchange
  - Initialize sender & receiver state
- Full duplex data:
  - Bi-directional data flow *
- Flow and congestion controlled:
  - Sender will not overwhelm receiver or network
- Reliable, in-order *byte stream:*
  - * No "message boundaries" *
    - App Layer looks like message, but writes bytes to socket
  - TCP segment
    - MSS: maximum segment size
- Send & Receive buffers
- Pipelined:
  - Variable window size set by TCP congestion and flow control

12

Full Duplex

13

TCP header – every segment



This header accompanies every single TCP segment transferred through the network.

16

Reliable In-order Byte stream – no message boundary

App Layer writes 40 bytes → TCP

17

MSS vs. MTU *

- MSS – Maximum Segment Size
- Maximum Transmission Unit

19

Reliable data transfer in TCP

* Cumulative ACKS, slightly different from GoBackN

22

Reliable data transfer in TCP

Sequence Numbers example

24

25

TCP seq. numbers, ACKs

Sequence Numbers:

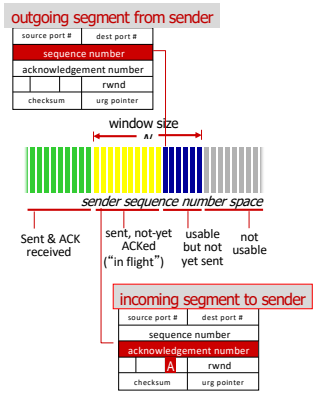
- Byte stream
- "number" of first byte in segment

Acknowledgements:

- Cumulative ACK
  - SN of next next byte expected by Receiver
  - "Send me SN = X"

Q: How does receiver handle out-of-order segments?

- *A:* TCP spec doesn't say! - up to implementation
  - Most likely receiver buffering
  - Does Sender know what Receiver is doing?



27

TCP sequence numbers, ACKs

Study for Quiz

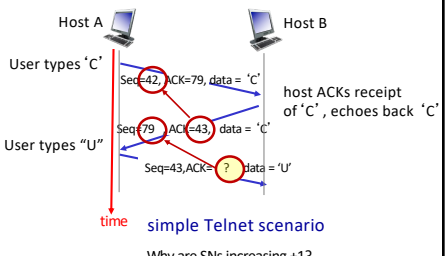
Seq. #'s:

- Byte stream "number" of first byte in segment

Cumulative ACK:

- Seq # of next byte expected from other side

Each segment carries 1 byte of data → byte representation for 'C'



simple Telnet scenario

Why are SNs increasing +1?
→Each segment carries 1 byte of data

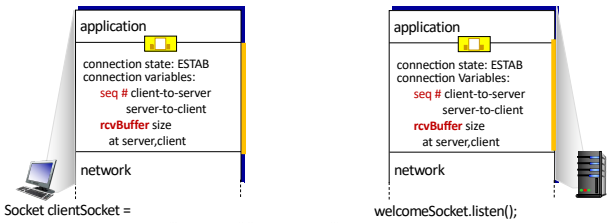
ACK = a) 43 b) 79 c) 80 d) 81

28

TCP Connection Management

Before exchanging data, "handshake":

- Agree to communicate and set up "state"
  - Allocate resources, agree and acknowledge connection parameters
  - Establish state variables (*e.g.,*initial SN, rcvBuffer)

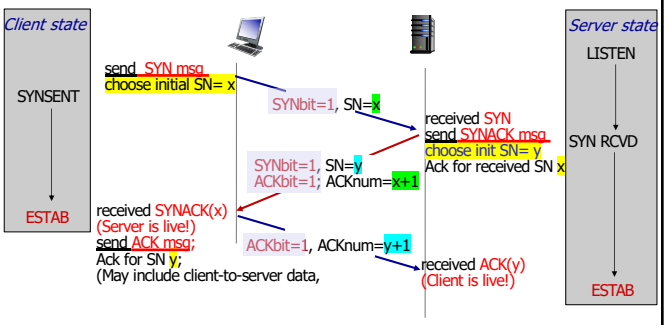


Socket clientSocket =
newSocket("hostname","port number");

welcomeSocket.listen();
Socket connectionSocket =
welcomeSocket.accept();

31

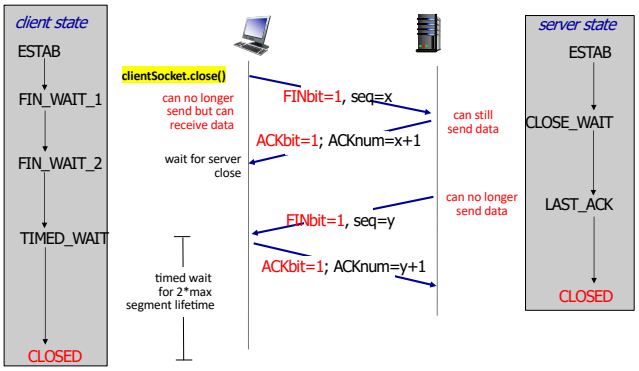
TCP open: 3-way handshake



Why does each side send initial SN?

32

TCP close: FIN exchange



Segment Lifetime = 2 min.

Details in book

34

TCP Reliable Data Transfer

- TCP creates rdt service over IP's best-effort service
  - Pipelined segments
  - Cumulative ACKs
  - Single retransmission timer
- Retransmissions triggered two ways:
  1.Timeout
  2.

Initially, consider simplified TCP sender:

- Ignore duplicate ACKs
- Ignore flow control, congestion control
- Just look at timeout and ACK arrival

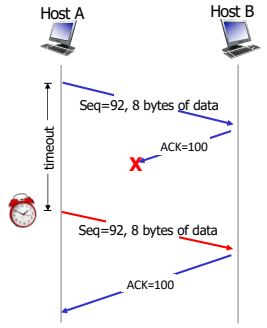
35

How do we set the Retransmission Timer?

- What should the value be?
- What happens if timer set too short or too long?
- 3 retransmission scenarios

37

TCP: retransmission scenarios
Scenario #1



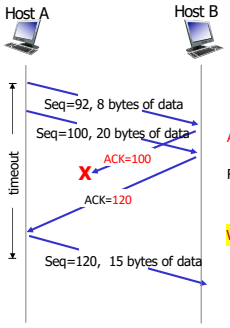
Why is there a timeout?

What happens at the Receiver?
1.
2.

lost ACK
Timeout -- no other segments in flight

38

TCP: retransmission scenarios
Scenario #2



ACK lost (no timeout); all data received

Reliable, in-order byte stream: what goes to App Layer?

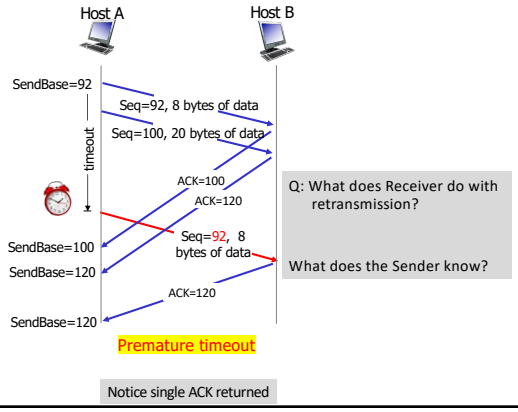
What do you think about the cumulative ACK?
Does it matter that earlier ACK was lost?

Lost ACK
(2nd ACK arrives before timeout)

Q: Is it necessary for the Sender to receive every single Cumulative ACK?

39

TCP: retransmission scenarios
Scenario #3



40

TCP Retransmission Timer

- What should the value be?
- What should be our considerations?

41

RTO: Retransmission Timeout

Features of RTO?

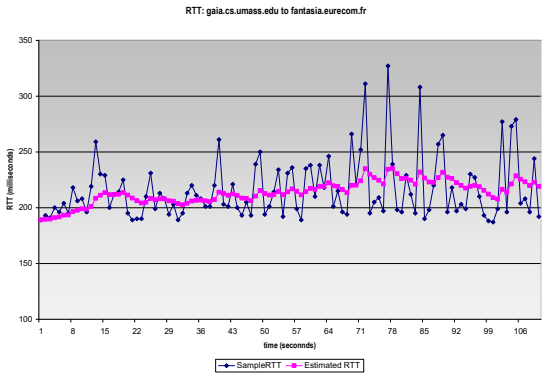
- longer than RTT, but what is that? (varies)
  - *too short:* premature timeout, unnecessary retransmissions
  - *too long:* slow reaction to loss

How to estimate RTT?

- SampleRTT: measured time interval from segment's transmission until ACK received
  - Ignore retransmissions
- samples vary (see graph)
- Want "smooth" RTT

42

Smooth Average! How to do it? (pink line)



Fluctuation in measured

43

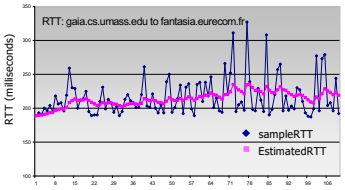
Sample RTT (AKA Measured)

- Take a sample

44

EstimatedRTT: incorporate each sample

- Technique: EWMA Exponential Weighted Moving Average
- Weight of older samples decrease as new are incorporated



Time (seconds)
Sometimes called "RTT round"

45

RTO Calculation: Step 1

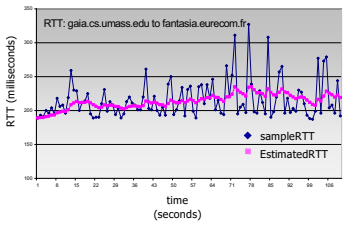
Step 1

EstimatedRTT = (1- α)*EstimatedRTT + α*SampleRTT

Typical value: α = 0.125

Think about values of α:

- α=1 ?
  reactive vs. "smoothing"
  "averaging"…
  Is emphasis on recent or past?



46

TCP RTO Calculation: Step 2 Step 3

Step 2

DevRTT: How much does SampleRTT deviate from EstimatedRTT?

DevRTT = (1-β) * DevRTT + β *|SampleRTT - EstimatedRTT|

(typically, β = 0.25)

Setting the Timeout (RTO) : EstimatedRTT plus "safety margin"

- large variation in **EstimatedRTT** → larger safety margin

Step 3 RTO = EstimatedRTT + 4 * DevRTT

"Safety Margin"

47

Types of ACKs

- ✓ Cumulative ACKs
- ✓ Piggybacked ACKs
- ✓ Duplicate ACKs (more soon)
- ✓ Delayed ACKs
  - Why useful?

48

TCP ACK generation [RFC 1122, RFC 2581]

Event at Receiver	TCP Receiver Action
Arrival of EXPECTED in-order segment with expected seq #	Delayed ACK. Wait up to 500ms for next segment. If nothing new arrives, send ACK
Arrival of in-order segment with expected seq # and one other segment has ACK pending	**Immediately** send single cumulative ACK (ACK covers both in-order segments)
-Arrival of out-of-order segment higher-than-expect seq. # -Gap detected	**Immediately** send duplicate ACK, indicating seq. # of next expected byte
Arrival of segment that partially or completely fills a gap	**Immediately** send cumulative ACK

49

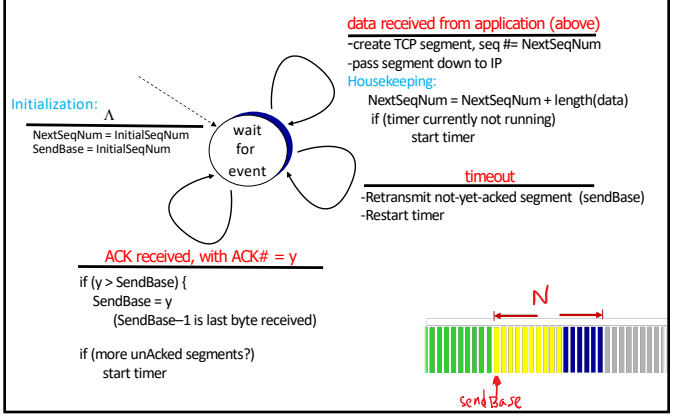
TCP Receiver: ACK generation [RFC 5681]

Event at receiver	*TCP receiver action / type of ACK*

Practice!

50

TCP Sender FSM(simplified)



51

Focus on Duplicate ACKs

During data transfer, what events occur at the Sender?

1.
2.
3.

What does duplicate ACK mean?

53

TCP Fast Retransmit – Early Loss Detection!

- Time-out period long:
  - retransmissions delayed
  - Sluggish performance
- Detect dropped segments quickly!
  - Pipelining
  - Dropped segments generate duplicate ACKs
  - Interpret the ACKs

TCP Fast Retransmit

If sender receives 3 duplicate ACKs

("triple duplicate ACKs")

likely a segment lost -- don't wait for timeout

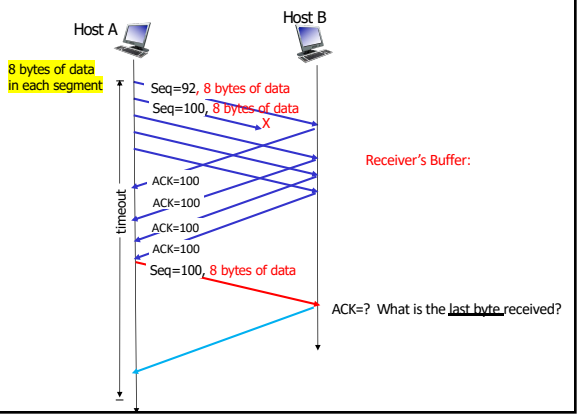
→ Retransmit!

54

TCP Fast Retransmit

after sender receives triple duplicate ACK

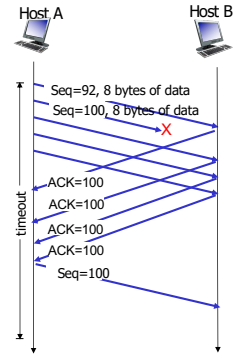
→"likely" segment is lost so Retransmit!



55

TCP Fast Retransmit – what have we gained?

How long is the RTO? See section 3.5.3



Long wait avoided... →

57

What does Fast Retransmit Solve?

- Treat symptoms of congestion in network
- Is it a solution?
- Stay tuned for TCP Congestion Control!

61

TCP Services

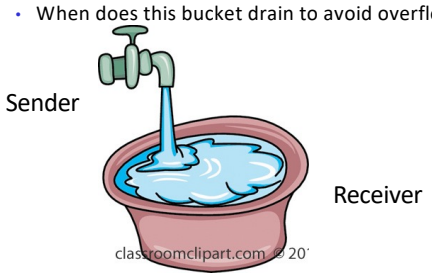
- ✓ Error control
  - ✓ Reliable data transfer: reliable, in-order delivery with duplicate detection
- Flow control
- Congestion control

In reality, they work together but we will be looking at them separately

63

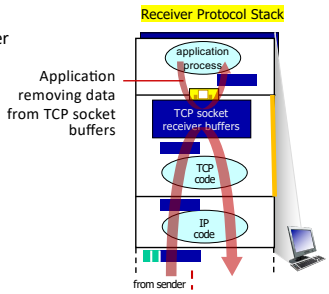
TCP Flow Control

- Goal:
  - Pace Sender to not overwhelm Receiver.
  - When does this bucket drain to avoid overflow?



TCP Flow Control

Q: What happens if network layer delivers data to transport layer faster than application layer removes data from receiver socket buffers?



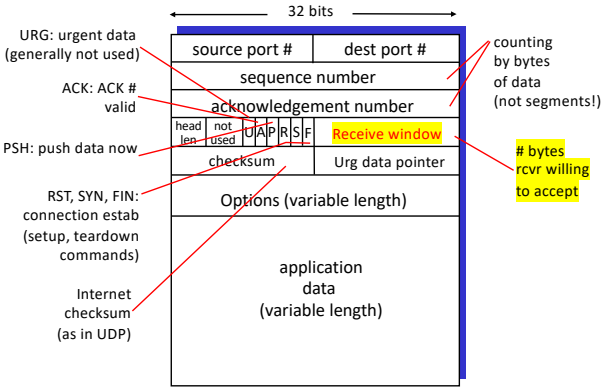
Flow control

Receiver controls sender! Sender won't overflow receiver's buffer by transmitting too much, too fast.

Speed-matching service:

send rate = receiving app's read rate

TCP segment structure



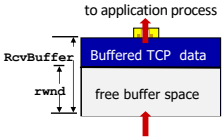
Flow versus congestion control

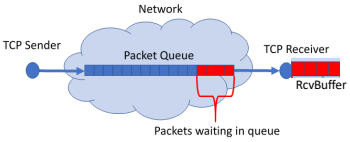
Goal: Don't overwhelm, but keep it coming!

- TCP Receiver "advertises" free buffer space in TCP header
- Sender limits its window size accordingly

  Sender's window <= rwnd

  where *rwnd* = RcvBuffer - [LastByteRcvd – LastByteRead]
- Guarantees receiver's buffer will not overflow





What is the difference, flow vs. congestion control?

- Pacing transmissions – different target

Image source: Argon Systems